Numeric Escape Sequences

There are also escape sequences for setting character byte values directly, using octal or hexadecimal notation:

```
print "\110\145\154\154\157 " .
    "\x57\x6f\x72\x6c\x64" . PHP_EOL;
```

```
Output:
Hello World
```

Octal escapes take the form `"\N"` where `N` is a valid octal value for an 8-bit byte; hexadecimal escapes are similar: `"\xN"` where `N` is a valid 8-bit hexadecimal value. (If you're not sure how to use numbers in base 8 or 16, see Chapter 7, "Numbers," for a detailed explanation.) The "Hello" portion of the output was produced with escaped octal values. Octal *110* is *72* in decimal, and codepoint *72* in the ASCII codepage is the capital letter "H," octal *145* is lowercase "e," and so on. "World" was produced in a similar way: hexadecimal *0x57* is ASCII capital "W," etc.

Because these are 8-bit bytes, the legal range of values for these escape sequences is octal *0–377*, hexadecimal *0x00–0xFF*. It is possible for an octal escape to overflow the maximum value (because the highest three-digit octal number is *777*). If that happens, a warning error will be emitted, and the byte value will be set to *0x00*:

```
var_dump(bin2hex("\400")); // octal 377 + 1
```

```
Output:
Warning: Octal escape sequence overflow \400 is greater than \377
string(2) "00"
```

Overflow is not possible with hexadecimal escapes because the maximum number of digits after the `\x` sequence is two:

```
var_dump(bin2hex("\x100")); // hex FF + 1
```

```
Output:
string(4) "1030"
```

Here we can see that the string `"\x100"` has been parsed as byte value *0x10* and the ASCII character "0" (byte *0x30* in the codepage). This also serves to demonstrate that you can quite happily mix-and-match numeric escape sequences with regular text, just as we did with the special character escapes above.